



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

JAKARTA EE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on CDI, JAX-RS, EJB, JMS, and enterprise security

TOTAL: 10 QUESTIONS

Q1 What is Jakarta EE and how does it relate to Java EE? [Threat Model](#)

Q6 How do you configure JPA persistence in a Jakarta EE application? [Observability](#)

Q2 What is CDI and what are its main bean scopes? [Architecture](#)

Q7 What is JNDI and how do you look up a resource by name? [Lifecycle](#)

Q3 How do you build a RESTful endpoint with JAX-RS? [Configuration](#)

Q8 How do Jakarta EE security mechanisms protect a JAX-RS endpoint? [Resilience](#)

Q4 When would you use a @Stateless EJB vs @Stateful vs @Singleton? [Hardening](#)

Q9 What is the difference between WAR, EAR, and executable JAR packaging? [Tools](#)

Q5 How does JMS work and what is the difference between Queue and Topic? [Queue](#)

Q10 How do CDI interceptors differ from EJB interceptors? [Tuning](#)



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Jakarta EE is the community-governed evolution of

Mitigation

Jakarta EE is the community-governed evolution of Java EE after Oracle transferred stewardship to the Eclipse Foundation. The namespace changed from `javax.*` to `jakarta.*` in EE 9. It standardizes enterprise APIs like CDI, JAX-RS, JPA, and JMS.

A6 Define META-INF/persistence.xml listing the persistence unit,

Define META-INF/persistence.xml listing the persistence unit, JTA data source, and entity classes. Inject the EntityManager with `@PersistenceContext(unitName='myPU')`. The container manages transactions, so use `@Transactional` or container-managed TX.

A2 CDI (Contexts and Dependency Injection) manages the

Primitives

CDI (Contexts and Dependency Injection) manages the lifecycle and injection of beans. `@RequestScoped` creates one instance per HTTP request. `@SessionScoped` per HTTP session. `@ApplicationScoped` is a singleton shared across the application.

A7 JNDI (Java Naming and Directory Interface) is

Automation

JNDI (Java Naming and Directory Interface) is a registry for server resources. `@Resource(lookup='java:comp/env/jdbc/myDS')` injects a DataSource directly. Programmatic lookup uses `InitialContext ctx = new InitialContext(); ctx.lookup('java:...')`.

A3 Annotate the class with @Path('/users') and method

Hardening

Annotate the class with `@Path('/users')` and method with `@GET/@POST` plus `@Produces(MediaType.APPLICATION_JSON)`. `@PathParam`, `@QueryParam`, and `@RequestBody` bind URL parts and request data. Extend Application or use a Jakarta EE server's auto-scan.

A8 @RolesAllowed('ADMIN') on a resource method restricts

Performance

`@RolesAllowed('ADMIN')` on a resource method restricts access to principals with that role. The security domain, identity store (`@DatabaseIdentityStoreDefinition`), and authentication mechanism (`@FormAuthenticationMechanismDefinition`) are configured via annotations.

A4 @Stateless handles requests without client-specific state and

Scalability

`@Stateless` handles requests without client-specific state and is pooled ideal for services. `@Stateful` maintains conversational state per client session use for shopping carts. `@Singleton` has one instance per JVM and supports concurrent access control.

A9 A WAR is a web archive deployed

Assessment

A WAR is a web archive deployed to a servlet container or app server. An EAR bundles multiple WARs and EJB JARs for a full app server. Modern Jakarta EE apps often use executable JARs with embedded servers (WildFly Bootable JAR, Open Liberty) for cloud deployment.

A5 A JMS Queue delivers each message to

Federation

A JMS Queue delivers each message to exactly one consumer (point-to-point). A Topic delivers the message to all subscribers (publish-subscribe). `@MessageDriven` EJBs or JMS listeners process messages asynchronously from the broker.

A10 CDI interceptors are bound to beans via

Optimization

CDI interceptors are bound to beans via `@InterceptorBinding`, a custom annotation that marks methods to intercept. EJB interceptors use `@Interceptors(MyInterceptor.class)` on the bean class. CDI interceptors work on any CDI-managed bean, not just EJBs.